

```

# -*- coding: utf-8 -*-
"""
Kyle Urban
University of Central Arkansas
Capstone File #1

"""
from dataclasses import dataclass
from copy import deepcopy
import math
import torch #May 31st, 2025 update, added in PyTorch cuz https://tenor.com/search/spongebob-i-need-it-gifs
import json
import os
import numpy as np #very important to the sanity check in part 2
import pandas as pd
#print(pd.__version__)
import tensorflow as tf
#print(tf.__version__)
from transformers import TFBertModel
from transformers import BertTokenizer

# Load the tokenizer
tokenizer = BertTokenizer.from_pretrained('bert-base-uncased')
bert_model = TFBertModel.from_pretrained("bert-base-uncased", from_pt = True) #Update as of 4/2/25 # 8/8 up

#move model onto GPU runtime to go faster (9/3/2025 update, ya' know ya boi got that google colabs pro)
''' #this is for pytorch only, tensorflow does this automatically
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
bert_model.to(device)
'''

#check if Tensorflow is finding the GPU's

print(tf.config.list_physical_devices('GPU'))

# Test tokenization
text = "Hello, TensorFlow!"
tokens = tokenizer.encode(text, return_tensors="tf") # Use "tf" instead of "pt" for TensorFlow tensors

@dataclass
class Desword_Pairs:
    row_num: int
    sentence1_des: str
    sentence2_des: str
    gold_label: str
    tokenized_s1: any = None
    tokenized_s2: any = None
    flatten_tok_s1: any = None
    flatten_tok_s2: any = None
    pad_flat_tok_s1: any = None
    pad_flat_tok_s2: any = None
    tensorized_s1: any = None
    tensorized_s2: any = None
    pad_tensorized_s1: any = None
    pad_tensorized_s2: any = None

```

```
embed_s1: any = none
```

```
embed_s2: any = None
```

```
last_hid_state_s1: any = None
```

```
last_hid_state_s2: any = None
```

```
cls_embed_1: any = None
```

```
cls_embed_2: any = None
```

```
def __init__(self, row_num ,sentence1_des ,sentence2_des , gold_label , tokenized_s1 , tokenized_s2 ,
              flatten_tok_s1 , flatten_tok_s2 , pad_flat_tok_s1 , pad_flat_tok_s2 , tensorized_s1, tensor
              pad_tensorized_s1 , pad_tensorized_s2 , embed_s1 , embed_s2 , last_hid_state_s1 ,
              last_hid_state_s2 , cls_embed_1 , cls_embed_2):
    self.row_num = row_num
    self.sentence1_des = sentence1_des
    self.sentence2_des = sentence2_des
    self.gold_label = gold_label
    self.tokenized_s1 = tokenized_s1
    self.tokenized_s2 = tokenized_s2
    self.flatten_tok_s1 = flatten_tok_s1
    self.flatten_tok_s2 = flatten_tok_s2
    self.pad_flat_tok_s1 = pad_flat_tok_s1
    self.pad_flat_tok_s2 = pad_flat_tok_s2
    self.tensorized_s1 = tensorized_s1
    self.tensorized_s2 = tensorized_s2
    self.pad_tensorized_s1 = pad_tensorized_s1
    self.pad_tensorized_s2 = pad_tensorized_s2
    self.embed_s1 = embed_s1
    self.embed_s2 = embed_s2
    self.last_hid_state_s1 = last_hid_state_s1
    self.last_hid_state_s2 = last_hid_state_s2
    self.cls_embed_1 = cls_embed_1
    self.cls_embed_2 = cls_embed_2
```

#the Desword_pair object also needs it own flatten to list obbject

#may not actually need this object

```
def flatten_tf_to_list(self):
    self.flatten_tok_s1 = self.tokenized_s1.numpy().flatten().tolist() #each line gets flattened
    self.flatten_tok_s2 = self.tokenized_s2.numpy().flatten().tolist() #each line gets flattened
```

```
def convert_back_to_tensors(self):
    print("takes flattened tensors and makes them unflat")

    self.tensorized_s1 = tf.reshape(tf.convert_to_tensor(self.flatten_tok_s1, dtype=tf.int32), (1, len(self.
    self.tensorized_s2 = tf.reshape(tf.convert_to_tensor(self.flatten_tok_s2, dtype=tf.int32), (1, len(self.
```

```
def pad_tensors(self, max_seq_len_ten_s1, max_seq_len_ten_s2):
    print("post pads all my tensors")
```

```
self.pad_tensorized_s1 = tf.keras.preprocessing.sequence.pad_sequences(self.tensorized_s1, maxlen = max_
self.pad_tensorized_s2 = tf.keras.preprocessing.sequence.pad_sequences(self.tensorized_s2, maxlen = max_
#https://www.tensorflow.org/api_docs/python/tf/keras/utils/pad_sequences
#rework the code here to get a maximum lenght atwhich the sequences can be padded 8/24/2025
```

```
print("row number: ", self.row_num)
print("pad_tensorized_s1: ", self.pad_tensorized_s1)
print("pad_tensorized_s2: " , self.pad_tensorized_s2)
```

```

def get_shape_tok_s1(self):
    return self.tokenized_s1.shape

def get_shape_tok_s2(self):
    return self.tokenized_s2.shape

def get_shape_tensorized_s1(self):
    return self.tensorized_s1.shape

def get_shape_tensorized_s2(self):
    return self.tensorized_s2.shape

def get_shape_pad_ten_s1(self):
    return self.pad_tensorized_s1.shape

def get_shape_pad_ten_s2(self):
    return self.pad_tensorized_s2.shape

#necessary for the word_search and removal method
def copy_deep(self):
    return deepcopy(self)

def word_search_and_remove(self, words_remove_tok_ids):
    print(self.row_num)

    #gonna have to do this 3 times. Once for modifying the p/cs1, once for modifying the h/cs2, and then once
    #this should return 6 sets total I believe

    #in order to do this imma need to make deep copies of every self (every entry) 3 times inside here.

    #removing from the deep copied sets
    removal_set = set(words_remove_tok_ids)

    #https://realpython.com/python-enumerate/
    #https://www.w3schools.com/python/ref_list_pop.asp
    #https://docs.python.org/3/tutorial/datastructures.html#list-comprehensions
    #https://stackoverflow.com/questions/1207406/how-to-remove-items-from-a-list-while-iterating

    #Will the number of sentences that had a preposition removed be the same as the number of sentences that
    #go over your code logic with Dr Stine.

    #p' --> cs2 and cs1' --> h
    self_s1mod = self.copy_deep()
    self_s1mod.flatten_tok_s1 = [token for token in self.flatten_tok_s1 if token not in removal_set]

    #makes sense because when working cs1h, not all cs1s will have preps
    if (len(self_s1mod.flatten_tok_s1) == len(self.flatten_tok_s1)):
        print("len of s1 ")
        print(len(self.flatten_tok_s1))
        print("len of s1' ")
        print(len(self_s1mod.flatten_tok_s1))

    #p --> cs2' and cs1 --> h'

```

```

self_s2mod = self.copy_deep()
self_s2mod.flatten_tok_s2 = [token for token in self.flatten_tok_s2 if token not in removal_set]

#makes sense because when working on pcs2, not all cs2s will have preps
if (len(self_s2mod.flatten_tok_s2) == len(self.flatten_tok_s2)):
    print("len of s2 ")
    print(len(self.flatten_tok_s2))
    print("len of s2' ")
    print(len(self_s2mod.flatten_tok_s2))

#p'--> cs2' and cs1' --> h'
self_bothmod = self.copy_deep()
self_bothmod.flatten_tok_s1 = [token for token in self.flatten_tok_s1 if token not in removal_set]
self_bothmod.flatten_tok_s2 = [token for token in self.flatten_tok_s2 if token not in removal_set]

return self_s1mod, self_s2mod, self_bothmod

def embed_the_padded_tensors(self):

    self.embed_s1 = bert_model(self.pad_tensorized_s1)
    self.embed_s2 = bert_model(self.pad_tensorized_s2)

def last_hid_state_from_embed(self):

    self.last_hid_state_s1 = self.embed_s1.last_hidden_state
    self.last_hid_state_s2 = self.embed_s1.last_hidden_state

def cls_embed_from_last_hid_state(self):

    self.cls_embed_1 = self.last_hid_state_s1[:,0,:]
    self.cls_embed_2 = self.last_hid_state_s2[:,0,:]

def get_cls_embeds_s1(self):
    return self.cls_embed_1

def get_cls_embeds_s2(self):
    return self.cls_embed_2

def get_cls_embeds_rownum(self):
    return self.row_num, self.cls_embed_1, self.cls_embed_2

```

```

#
#
#

```

@dataclass #this makes the __init__ and __repr__ functions for me :) (took this line from ChatGPT)

```
class MultiNLI_entry:
    row_num: int
    sentence1: str
    sentence2: str
    gold_label: str
    tokenized_s1: any = None
    tokenized_s2: any = None
    flatten_tok_s1: any = None
    flatten_tok_s2: any = None
    pad_flat_tok_s1: any = None
    pad_flat_tok_s2: any = None
    tensorized_s1 : any = None
    tensorized_s2: any = None
    pad_tensorized_s1: any = None
    pad_tensorized_s2: any = None
    embed_s1: any = None
    embed_s2: any = None
    last_hid_state_s1: any = None
    last_hid_state_s2: any = None
    cls_embed_1: any = None
    cls_embed_2: any = None

    def flatten_tf_to_list(self):
        self.flatten_tok_s1 = self.tokenized_s1.numpy().flatten().tolist() #each line gets flattened
        self.flatten_tok_s2 = self.tokenized_s2.numpy().flatten().tolist() #each line gets flattened

    def word_search(self, words_remove_tok_ids):
        sent1_has_word = False
        sent2_has_word = False

        removal_set = set(words_remove_tok_ids)
        for token in self.flatten_tok_s1:
            if token in removal_set:
                sent1_has_word = True

        for token in self.flatten_tok_s2:
            if token in removal_set:
                sent2_has_word = True

        return sent1_has_word, sent2_has_word

    def copy_to_desword_pair(self):
        copied = deepcopy(self)

        return Desword_Pairs( row_num = copied.row_num,
                               sentence1_des= copied.sentence1,
                               sentence2_des = copied.sentence2,
                               gold_label = copied.gold_label,
                               tokenized_s1 = copied.tokenized_s1,
                               tokenized_s2 = copied.tokenized_s2,
                               flatten_tok_s1 = copied.flatten_tok_s1,
                               flatten_tok_s2 = copied.flatten_tok_s2
```

```

flatten_tok_s2 = copied.flatten_tok_s2,
pad_flat_tok_s1 = copied.pad_flat_tok_s1,
pad_flat_tok_s2 = copied.pad_flat_tok_s2,
tensorized_s1 = copied.tensorized_s1,
tensorized_s2 = copied.tensorized_s2,
pad_tensorized_s1 = copied.pad_tensorized_s1,
pad_tensorized_s2 = copied.pad_tensorized_s2,
embed_s1 = copied.embed_s1,
embed_s2 = copied.embed_s2,
last_hid_state_s1 = copied.last_hid_state_s1,
last_hid_state_s2 = copied.last_hid_state_s2,
cls_embed_1 = copied.cls_embed_1,
cls_embed_2 = copied.cls_embed_2 )

```

#gonna have a desword pairs that stores the sentence 1's with preps and the sentence 2 that matches to them
 #basically the P --> CS2 and variations of
 #the another that has the sentence 2's and the sentence 1 that matches to them
 #basically the CS1 --> H and variations of
 #the removal of preps (or other words) shall take place as an external functions, and the variations of the
 #shall be stored as such in the sentence1_des and sentence2_des of appropriately labeled arrays of objects

```

def prep_data():
    print("prep data")

    #name of the multinli jsonl file
    file_name = "multinli_1.0_dev_matched.jsonl"

    entries = []

    with open(file_name, "r", encoding="utf=8") as file:
        for index, line in enumerate(file):
            data_line = json.loads(line)
            entry = MultiNLI_entry(
                row_num = index,
                sentence1 = data_line.get("sentence1", ""),
                sentence2 = data_line.get("sentence2", ""),
                gold_label = data_line.get("gold_label", ""),
            )

            entries.append(entry)

    return entries

#I could have made this a method
def tokenize_data(entries, tokenizer):
    print("tokenize data")

    for entry in entries:
        entry.tokenized_s1 = tokenizer.encode(entry.sentence1, return_tensors = "tf")
        print(entry.tokenized_s1)
        entry.tokenized_s2 = tokenizer.encode(entry.sentence2, return_tensors = "tf")
        print(entry.tokenized_s2)

def flatten_data(entries, tokenizer):
    print("calls class MultiNLI method to flatten data")

    for entry in entries:
        entry.flatten_tf_to_list()

```

```

    #print(type(entry.flatten_tok_s1))
    #print(type(entry.flatten_tok_s2))

def desired_words_to_remove(tokenizer):
    #for now this is hard_coded in, later I want this to be a text file that you can insert for custom words
    print("desired words to be removed")

    words_to_remove = ["above", "below", "behind", "inside", "outside"]

    words_remove_tok_ids = [tokenizer.encode(word, add_special_tokens = False)[0] for word in words_to_remove]

    print("Token Ids of words to remove: ", words_remove_tok_ids)
    return words_remove_tok_ids

#finish this later

#remove_words_token_ids = {key: tokenizer.encode(key, add_special_tokens = False)}

def find_sentences_with_only_desired_words_to_remove(words_remove_tok_ids, all_nli_entries):
    print("this should find only the sentences with the desired words to remove and store them in a new array
    #entries but only the entries that contain the desired words to be removed
    entries_deswords_s1 = [] #where s1 has the entries we want and s2 is just along for ride
    entries_deswords_s2 = [] #where s2 has the entries we want and s1 is just along for ride

    counter_s1 = 0
    counter_s2 = 0

    #marks true or false for any entry in all the nli data if there is a sentence with a preposition
    for entry in all_nli_entries:
        sent1_has_word, sent2_has_word = entry.word_search(words_remove_tok_ids)
        #print(sent1_has_word, sent2_has_word)

        #for the
        if sent1_has_word: #and not sent2_has_word:
            #print("place holder")
            new_desword_pair = entry.copy_to_desword_pair() #is this how I do it? #8/15 update, I added the parentl
            entries_deswords_s1.append(new_desword_pair)
            #deep copy any sent1 with words to desword pair
            counter_s1 += 1

        if sent2_has_word: #and not sent1_has_word:
            #print("place holder")
            new_desword_pair = entry.copy_to_desword_pair()
            entries_deswords_s2.append(new_desword_pair)
            #deep copy any sent2 with words to desword pair
            counter_s2 += 1

    #as a test
    print(" p --> cs2")
    print(entries_deswords_s1[0])
    print(" cs1 --> h")
    print(entries_deswords_s2[0])

```

```

print("counter_s1: ", counter_s1 )
print("counter_s2: ", counter_s2)

return entries_deswords_s1, entries_deswords_s2, counter_s1, counter_s2

def remove_desired_words_from_sentences(pcs2, cs1h, words_remove_tok_ids, tokenizer):
    print("use the tokenizer to remove the tokens that correspond to the tokens of the words to remove")

    p_cs2 = []
    pcs2_ = []
    p_cs2_ = []

    cs1_h = []
    cs1h_ = []
    cs1_h_ = []

    for entry in pcs2:
        p_mod, cs2_mod, both_mod = entry.word_search_and_remove(words_remove_tok_ids)
        p_cs2.append(p_mod)
        pcs2_.append(cs2_mod)
        p_cs2_.append(both_mod)

    print("_____")

    for entry in cs1h:
        cs1_mod, h_mod, both_mod = entry.word_search_and_remove(words_remove_tok_ids)
        cs1_h.append(cs1_mod)
        cs1h_.append(h_mod)
        cs1_h_.append(both_mod)

    return p_cs2, pcs2_, p_cs2_, cs1_h, cs1h_, cs1_h_

def unflatten_tokens_back_to_tensors(data_set):
    for entry in data_set:
        entry.convert_back_to_tensors()

def pad_helper(data_set):

    max_seq_len_ten_s1, max_seq_len_ten_s2 = max_seq_len_finder(data_set)

    for entry in data_set:
        entry.pad_tensors(max_seq_len_ten_s1, max_seq_len_ten_s2) #hope this works 8/24/2025

def max_seq_len_finder(data_set):
    print("placeholder")

    all_seq_lens_in_data_set = max_seq_len_getter(data_set)

    max_seq_len_in_data_set_s1 = max(entry[3] for entry in all_seq_lens_in_data_set)
    max_seq_len_in_data_set_s2 = max(entry[4] for entry in all_seq_lens_in_data_set)

    return max_seq_len_in_data_set_s1, max_seq_len_in_data_set_s2

```



```

def max_seq_len_getter(data_set):
    all_seq_lens_in_data_set = []

    for entry in data_set:
        all_seq_lens_in_data_set.append(shape_getter(entry))

    return all_seq_lens_in_data_set

def shape_getter(entry):
    return ( entry.row_num,                #0
            entry.get_shape_tok_s1()[1],   #1
            entry.get_shape_tok_s2()[1],   #2
            entry.get_shape_tensorized_s1()[1], #3
            entry.get_shape_tensorized_s2()[1]) #4    #all the shapes are 2d, I only care about the second d:

def shape_getter_pad_ten(entry):

    return (entry.row_num,
            entry.get_shape_pad_ten_s1()[1],
            entry.get_shape_pad_ten_s2()[1])

def shape_checker_pad_ten(data_set):
    data_set_storage = []
    max_seq_len_in_data_set_s1, max_seq_len_in_data_set_s2 = max_seq_len_finder(data_set)

    for entry in data_set:
        data_set_storage.append(shape_getter_pad_ten(entry))

    #this gets the minimum sequence length (2d dimension for the padded)
    min_seq_len_in_data_set_s1 = min(entry[1] for entry in data_set_storage)
    min_seq_len_in_data_set_s2 = min(entry[2] for entry in data_set_storage)

    #since this was padded, the min_seq_length should be the same as the max_seq_length

    if max_seq_len_in_data_set_s1 == min_seq_len_in_data_set_s1:
        print("the padding worked")
    else:
        print("something fishy here")
        print(max_seq_len_in_data_set_s1, min_seq_len_in_data_set_s1)

    if max_seq_len_in_data_set_s2 == min_seq_len_in_data_set_s2:
        print("the padding worked")
    else:
        print("something fishy here")
        print(max_seq_len_in_data_set_s2, min_seq_len_in_data_set_s2)

    ...

    for entry in data_set_storage:
        print(f"data_set {data_set}, row_num: {entry[0]}, pad_tok_s1: {entry[1]}, pad_tok_s2: {entry[2]}")

    ...

def get_cls_embeds(data_set):

```

```

for entry in data_set:
    get_last_hid_state(entry)
    entry.cls_embed_from_last_hid_state()
    print(entry.get_cls_embeds_rownum())

def get_last_hid_state(entry):
    get_embeds(entry)
    entry.last_hid_state_from_embed()
    return

def get_embeds(entry):
    entry.embed_the_padded_tensors()
    return

def flatten_og_prior_to_copy(all_nli_entries):
    for entry in all_nli_entries:
        entry.flatten_tf_to_list()

def copy_og_onto_desword_class(all_nli_entries):

    og_in_desword = []

    for entry in all_nli_entries:
        og_pair = entry.copy_to_desword_pair()
        og_in_desword.append(og_pair)

    return og_in_desword, len(og_in_desword)

def main():
    print("start of program")
    all_nli_entries = prep_data()
    print(all_nli_entries)
    #tokenize_data function returns a tensorflow tensor
    tokenize_data(all_nli_entries, tokenizer)
    #remember that all_nli_entries is all 10000 sentences, if we are gonna search and rip prepositions
    #we need have two functions that search those, put them into another class MultiNLI, then another function
    #search and destroy the preps. Has to happen between tokenize and tensorize steps
    #the cls embeds for all the tensorized data could be used as training data over in the other file?
    flatten_data(all_nli_entries, tokenizer)
    words_remove_tok_ids = desired_words_to_remove(tokenizer)

    print(all_nli_entries[0])
    #at this point, pcs2 and cs1h should still be flattened, same as all_nli_entries 8/23/2025 note
    #a later function is needed to unflatten/convert them back to tensorflow tensors
    #actually wrong 8/23 note, but later, they are flattened inside the object?
    pcs2, cs1h, word_counter_s1, word_counter_s2 = find_sentences_with_only_desired_words_to_remove(words_remove_tok_ids)
    flatten_og_prior_to_copy(all_nli_entries) #important this step happens before the copy
    og_set, len_og_set = copy_og_onto_desword_class(all_nli_entries)
    print(len_og_set)
    print(og_set[0].flatten_tok_s1)
    print(og_set[0].flatten_tok_s2)

```

```
print(og_set[0].flatten_tok_s2)
```

```
#very important make sure to uncomment (9/3/2025 -- 2:03pm)
#commented out for now to work on OG
```

```
print(pcs2[0].flatten_tok_s1)
print(pcs2[0].flatten_tok_s2)
```

```
print(cs1h[0].flatten_tok_s1)
print(cs1h[0].flatten_tok_s2)
```

```
#word_counter_s1 and word_counter_s2 should match the counter coming out the removed
#preps method named word_search_and_remove of the Desired_Pairs class
#if they don't something went wrong
```

```
p_cs2, pcs2_, p_cs2_, cs1_h, cs1h_, cs1_h_ = remove_desired_words_from_sentences(pcs2, cs1h, words_remove_1
```

```
#these tokens are not truly unflattened!
#https://www.tensorflow.org/api_docs/python/tf/reshape you can reshape a tensors with sequence length
#all my tensors are (1, Xth dimension of sequence length)
#tf.reshape(tf.convert_to_tensor(dataset, dtype= tf.int32), (1, sequence_length))
unflatten_tokens_back_to_tensors(pcs2)
unflatten_tokens_back_to_tensors(p_cs2)
unflatten_tokens_back_to_tensors(pcs2_)
unflatten_tokens_back_to_tensors(p_cs2_)
unflatten_tokens_back_to_tensors(cs1h)
unflatten_tokens_back_to_tensors(cs1_h)
unflatten_tokens_back_to_tensors(cs1h_)
unflatten_tokens_back_to_tensors(cs1_h_)
```

```
unflatten_tokens_back_to_tensors(og_set) # 9/5/2025 update, unflattens og set
```

```
print(pcs2[0].tensorized_s1.shape)
print(pcs2[0].tokenized_s1.shape)
```

```
print(og_set[0].tensorized_s1.shape)
print(og_set[0].tokenized_s1.shape)
```

```
print("pcs2: (max_seq_len_ten_s1, max_seq_len_ten_s2): ", max_seq_len_finder(pcs2))
print("p_cs2: (max_seq_len_ten_s1, max_seq_len_ten_s2): ", max_seq_len_finder(p_cs2))
print("pcs2_: (max_seq_len_ten_s1, max_seq_len_ten_s2): ", max_seq_len_finder(pcs2_))
print("p_cs2_: (max_seq_len_ten_s1, max_seq_len_ten_s2): ", max_seq_len_finder(p_cs2_))
print("cs1h: (max_seq_len_ten_s1, max_seq_len_ten_s2): ", max_seq_len_finder(cs1h))
print("cs1_h: (max_seq_len_ten_s1, max_seq_len_ten_s2): ", max_seq_len_finder(cs1_h))
print("cs1h_: (max_seq_len_ten_s1, max_seq_len_ten_s2): ", max_seq_len_finder(cs1h_))
print("cs1_h_: (max_seq_len_ten_s1, max_seq_len_ten_s2): ", max_seq_len_finder(cs1_h_))

print("og_set: (max_seq_len_ten_s1, max_seq_len_ten_s2): ", max_seq_len_finder(og_set))
```

```
#I need to find what the highest 2nd dimensional value of the shape of the tensorized_s1 and tensorized_s2
#so that I can use it to pad everything to a uniform length
#so I'll need to store (using the shape getter (it returns a tuple)) all the values into a an array that
```

```
#hold the second value and the rownum in a tuple.  
#then use a search algorithm  
#how does python divide a tuple if I return three values
```

```
pad_helper(pcs2)  
pad_helper(p_cs2)  
pad_helper(pcs2_)  
pad_helper(p_cs2_)  
pad_helper(cs1h)  
pad_helper(cs1_h)  
pad_helper(cs1h_)  
pad_helper(cs1_h_)  
  
pad_helper(og_set)
```

```
shape_checker_pad_ten(pcs2)  
shape_checker_pad_ten(p_cs2)  
shape_checker_pad_ten(pcs2_)  
shape_checker_pad_ten(p_cs2_)  
shape_checker_pad_ten(cs1h)  
shape_checker_pad_ten(cs1_h)  
shape_checker_pad_ten(cs1h_)  
shape_checker_pad_ten(cs1_h_)
```

```
shape_checker_pad_ten(og_set) #added on 9/5/2025 at 9:47pm
```

```
get_cls_embeds(pcs2)  
get_cls_embeds(p_cs2)  
get_cls_embeds(pcs2_)  
get_cls_embeds(p_cs2_)  
get_cls_embeds(cs1h)  
get_cls_embeds(cs1_h)  
get_cls_embeds(cs1h_)  
get_cls_embeds(cs1_h_)
```

```
get_cls_embeds(og_set) #added on 9/5/2025
```

```
#I need to find what the highest 2nd dimensional value of the shape of the tensorized_s1 and tensorized_s2  
#so that I can use it to pad everything to a uniform length  
#so I'll need to store (using the shape getter (it returns a tuple)) all the values into a an array that  
#hold the second value and the rownum in a tuple.  
#then use a search algorithm  
#how does python divide a tuple if I return three values
```

```
#check if Tensorflow is finding the GPU's
```

```
print(tf.config.list_physical_devices('GPU'))
```

```
#pad after finding and removing
```

```
#retokenize
```

```
if __name__ == "__main__":  
    main()
```



```

8.90742987e-02, -1.36805236e-01, 2.02599481e-01,
3.02388191e-01, 1.10087395e+00, 2.35453993e-03,
4.12066467e-03, 3.38164896e-01, 8.75173286e-02,
6.29307628e-01, -1.84800744e-01, 7.45645642e-01,
1.50984928e-01, -1.84926122e-01, 4.21052486e-01,
1.52799979e-01, 8.68231714e-01, -3.04377884e-01,
-3.41221273e-01, -5.10595977e-01, -2.56911844e-01,
7.08496332e-01, -2.46767417e-01, -4.32442278e-01,
-7.95515835e-01, -2.04905979e-02, -1.52637318e-01,
2.86454409e-01, -1.14073718e+00, -7.28410959e-01,
-6.37618065e-01, 5.46825528e-01, 5.40416956e-01,
2.98240520e-02, 2.15980679e-01, 3.30508292e-01,
-1.84255138e-01, -7.92175949e-01, 2.95342058e-01,
-6.56593621e-01, 4.36026603e-02, -1.01041161e-01,
3.54672909e-01, -2.96228081e-01, -4.73050952e-01,
7.33633876e-01, 5.54679871e-01, 7.11012483e-01,
-3.87159467e-01, 8.17166805e-01, 5.95995903e-01,
8.96248460e-01, -1.84987724e-01, 7.35234916e-01,
-1.26002327e-01, 8.93017501e-02, -1.38899684e-02,
-4.82551008e-01, -1.20709717e-01, -5.63966691e-01,
-1.60881832e-01, -1.50534734e-01, -2.30100393e-01,
3.35510224e-01, -7.73006082e-01, -1.57743514e-01,
-4.02193904e-01, 3.82692248e-01, -7.10903823e-01]], dtype=float32)>)
[PhysicalDevice(name='/physical device:GPU:0', device_type='GPU')]

```

Start coding or [generate](#) with AI.